

Architectural Justification — Lab 3

Community Lost & Found Matching Network (Problem Statement #59)
SRN: PES1UG25CS808 | Bhuvan Raj H | Section B

Architecture Selection: We chose the **Layered Architecture** (Presentation, Business, Data) for the Community Lost & Found Matching Network.

Reason 1 — Clear separation for evolving matching logic: The system's core value lies in its matching logic (tag extraction, category/colour comparison, geo-proximity matching), which is expected to be refined over time. Isolating this in a dedicated Business Layer (Report & Matching Manager) means the matching algorithm can be improved without touching the Presentation Layer used by Finder/Owner and Community Admin, or the Data Layer schema.

Reason 2 — Two actor types share one consistent rule set: Both the Finder/Owner and Community Admin interact with the same underlying reports and claims. A layered structure lets both the ordinary User Interface flows and the Admin Management flows route through the same Business Layer, guaranteeing that matching, verification, and moderation rules are applied consistently regardless of which actor is interacting with the system.

Security Advantage: FR004/NFR002 require that a claimant's contact details stay hidden until proof-of-ownership verification succeeds. In a layered design, the Presentation Layer never has direct access to the Data Layer — every request must pass through the Business Layer's Verification Service. This forces the ownership-verification check to sit on the only path to the Database Component, so contact details cannot be exposed by a shortcut in the UI.

Performance Benefit: NFR001 requires acceptable response times for matching under simulated peak load. Because geo-proximity and tag matching are computed within a single Business Layer process rather than being split across independently-deployed services (as in Microservices), there is no added network hop or serialization latency between the matching step and the verification step — keeping the report-to-match flow fast without the operational overhead a microservices split would add for a community-scale system.

Why not the alternatives: Microservices would add network-latency and deployment complexity disproportionate to this community-scale system's load. A pure Client-Server split (without internal layering) would still centralize control, but would not enforce the same clean separation between presentation, matching/verification logic, and storage — making it harder to guarantee that contact details are only released after verification.